

Hence, this implies that there are operations in T_1 that are not contained in the same version number:

$$\exists o_1, o_2 \in T_1, tbl(o_1) = tbl(o_2), v(o_1) \neq v(o_2) \quad (3)$$

This contradicts [Definition 3.1](#) as $T_1 \in T$. Hence, T_2 can only have access to none or all operations of T_1 affecting the same table. This implies the table-level atomicity of OTFs. $\square$

Lemma 3.1. *OTFs guarantee table-level atomicity by design. Due to their restriction to single-query and single-table transactions, they uphold this guarantee for all their transactions.*

3.2 Atomicity of LV [R]

LV [R] builds on [Lemma 3.1](#) to guarantee the same table-level atomicity guarantee. However, its markers allow LV [R] to drop the single-query constraint.

Definition 3.2. LV [R] uses markers to reserve a specific table version v for a transaction until its commit.

Direct Proof: Like the original OTFs, LV [R] determines the read version of a table and calculates its expected commit version in the following way:

$$\text{marker_version}(t) = \text{read_version}(t) + 1, t \in T \quad (4)$$

Unlike OTFs, LV [R] transactions reserve this specific version as a marker. If it fails to acquire the particular marker (due to concurrency and the atomic request to object store failing), LV [R] will never continue but continue to read and claim a respective marker. When it succeeds, it can be sure that the transaction is the only one using this number (as default OTF clients won't be able to place their log under the same version either. During its commit, LV [R] replaces the marker with a log containing all its operations upholding [Definition 3.1](#). Hence, LV [R] publishes all its operations of a specific table simultaneously, as they are all in the same file. However, because we reserved the version at the beginning and can hold markers as long as needed, we drop the single-query restriction of OTFs and get:

Lemma 3.2. *LV [R] guarantees table-level atomicity by design. Due to its restriction to single-table transactions, it upholds this guarantee for all its transactions.*

$\square$

3.3 Atomicity of LV [CT]

LV [CT] places a marker in each accessed table, reserving its entire version combination. During the commit, an LV [CT] transaction separates its operations by the affected table and places it in the respective log, replacing the marker. Hence, it ensures [Lemma 3.2](#) is upheld for all affected tables. However, as all metadata layers are distinct, its atomicity guarantee is limited at the table level.

Lemma 3.3. *LV [CT] guarantees table-level atomicity by design. Due to the reliance on distinct metadata layers, atomicity can not be guaranteed globally.*

3.4 Atomicity of LV [I]

Direct Proof: LV [I] replaces the OTF version assignment. Hence, the decision of the accessed read version now depends on the global version log for all tables. To access another's dirty data, a transaction

must receive a partially committed version from the global version log. However, this is impossible because only one transaction can enter the validation phase (using atomic object store requests) and advance the global version log one with a single write request. A concurrent read either receives the complete previous version or the completely committed one.

Lemma 3.4. *LV [I] guarantees atomicity globally.*

$\square$

4 Consistency

The database consistency of OTFs comes from their metadata layer design. Those formats define an ordering between operations by their metadata layer. For example, Delta Lake's logs form a list with strictly defined version numbers. The same holds for LakeVilla as it does not change the structure of the base OTF.

Lemma 4.1. *OTFs guarantee database consistency by defining a strict order among all transactions affecting a table.*

Lemma 4.2. *LV does not change the base OTF structure. Hence, [Lemma 4.1](#) also applies for LV.*

4.1 Distributed consistency of LV [R]

Direct Proof: Following [Definition 2.1](#), all transactions $t \in T$ define an order among its operations. Hence:

$$\forall t \in T, \forall x, y \in T, x \neq y : x \sim y \quad (5)$$

Due to LV [R]'s marker technique guarantees that all $ops \in t \in T$ affecting the same table are placed in the same metadata file ([Lemma 3.2](#)). Following [Lemma 4.1](#), we know that the OTF layer defines an order between transactions (e.g., in Delta Lake: log files). Thus:

$$\forall t_1, t_2 \in T, |tbl(t_1)| = 1, tbl(t_1) = tbl(t_2) : t_1 \sim t_2 \quad (6)$$

As LV [R] is applied to the base OTF, combining [Eq. \(5\)](#) and [Eq. \(6\)](#) gives:

$$\forall t_1, t_2 \in T, |tbl(t_1)| = 1, tbl(t_1) = tbl(t_2), x \in t_1, y \in t_2, x \neq y : x \sim y \quad (7)$$

As [Eq. \(5\)](#) defines $\sim$ if $t_1 = t_2$ and [Eq. \(6\)](#) if $t_1 \neq t_2$. However, [Eq. \(7\)](#) only holds for single-table transactions affecting the same table ($|tbl(t_1)| = 1, tbl(t_1) = tbl(t_2)$). Hence, if $tbl(t_1) \neq tbl(t_2)$ or $|tbl(t_1)| > 1$ or $|tbl(t_2)| > 1$, we know $t_1 \not\sim t_2$. This implies table-level causal consistency for LV [R] as only the order of related operations is defined. This result was expected as LV [R] focuses on table-level guarantees and does not track dependencies between tables like the base OTFs $\square$

Lemma 4.3. *LV [R] guarantees causal consistency on a table level using its marker technique:*

$$\forall t_1, t_2 \in T, |tbl(t_1)| = 1, tbl(t_1) = tbl(t_2), x \in t_1, y \in t_2, x \neq y : x \sim y$$

4.2 Distributed consistency of LV [CT]

Direct Proof: LV [CT] reuses the marker technique also found in LV [R] but places them in each accessed table. The deadlock detection ensures that the order between transactions is the same across all accessed tables. Hence:

Lemma 4.4. *LV [CT] deadlock detection and marker shift guarantee the same transaction order across all tables:*

$$\forall t_1, t_2 \in T, t_1 \neq t_2, \forall i_1, i_2 \in \text{tbl}(t_1), i_1, i_2 \in \text{tbl}(t_2) : v_{i_1}(t_1) \sim v_{i_1}(t_2) \Leftrightarrow v_{i_2}(t_1) \sim v_{i_2}(t_2)$$

Additionally, Eq. (5) applies for each individual LV [CT] transaction and implies relations between operations of the same transaction that do not affect the same table. Following this logic, we can apply Eq. (5) for each transaction separately on Lemma 4.4, we get:

$$\forall t_1, t_2 \in T, \text{tbl}(t_1) \cap \text{tbl}(t_2) \neq \emptyset, x \in t_1, y \in t_2, x \neq y : x \sim y \quad (8)$$

Comparing this to Eq. (7) shows that Lemma 4.4 effectively removed the restriction to single table transactions $|\text{tbl}(t_1)| = 1$. However, as LV [CT] does not define an order between write transactions not working on the same tables ($\text{tbl}(t_1) \cap \text{tbl}(t_2) = \emptyset$), we know that LV [CT] guarantees eventual consistency across all tables. $\square$

Lemma 4.5. *LV [R] guarantees causal consistency:*

$$\forall t_1, t_2 \in T, \text{tbl}(t_1) \cap \text{tbl}(t_2) \neq \emptyset, x \in t_1, y \in t_2, x \neq y : x \sim y$$

4.3 Distributed consistency of LV [I]

Direct proof: Again, we use Eq. (5) to show that transactions already define an order between all their operations. Hence, we must show that LV [I] defines an order among all writes. LV gives this [I]'s validation phase: Only one transaction at a time can enter the validation phase of the global version log. All table-level operations are written in the respective metadata layers before the global commit (compare Lemma 3.4) if it succeeds. This automatically places the transaction logically before all non-committed transactions. Hence:

Lemma 4.6. *LV [I]'s atomic validation defines a global relation among all committed global LV [I] transactions of the form:*

$$t_1 < t_2 < \dots < t_i$$

However, this does not include non-LV [I] transactions. Such transactions bypass the global version log and are only noticeable after they place a log (base OTF) or marker (LV [R], LV [CT]) in a table. As these claimed versions are not registered in the global version log, the validation of an LV [I] transaction will succeed. Still, its commit procedure will not as it discovers conflicting versions. The version log will not publish the validated transaction but will integrate those versions. Hence, it orders such transactions T_{other} by their appearance in tables (which is strictly defined; e.g., Eq. (6)) after the last validated transaction $t_{LV[I]}$:

$$\forall t \in T_{\text{other}} : t_{LV[I]} < t \quad (9)$$

The failed LV [I] transaction triggering this transaction must proceed with a new validation phase. However, this is sometimes avoided in our implementation with LV [I]'s concurrent hidden version checks. Hence, LV [I] defines a global order between all transactions:

Lemma 4.7. *LV [I]'s atomic validation and concurrent integration define a global relation among all committed transactions of the form:*

$$t_1 < t_2 < \dots < t_i$$

Again, combining Lemma 4.7 with Eq. (5) for each transaction in T gives:

$$\forall t_1, t_2 \in T, x \in t_1, y \in t_2, x \neq y : x \sim y \quad (10)$$

However, concurrent read-only transactions are an exception. Especially if they do not use markers or LV [I]'s WriteSerializable mode, they get table versions assigned at the beginning and skip any noticeable commit procedure. Depending on their required work, they might commit considerably later than concurrent write transactions. Hence, as they do not affect metadata layers, they must be logically ordered before concurrent writes to be consistent. Thus, LV [I] still defines an order among all transactions but does not preserve the commit order. This implies linearizability. $\square$

Lemma 4.8. *LV [I] guarantees linearizability for all transactions:*

$$\forall t_1, t_2 \in T, x \in t_1, y \in t_2, x \neq y : x \sim y$$

5 Isolation

For this section, we will use the following definition:

Lemma 5.1. *A history is serializable if its outcome matches a sequential execution of all transactions.*

As OTFs define a table-level order of transactions (Eq. (6)) and each transaction an order among its operations (Eq. (5)), we can link the serializability with its metadata orderings:

Lemma 5.2. *Serializable $\Leftrightarrow$ the OTF transaction order is acyclic.*

5.1 Isolation of OTFs

To investigate LV's isolation levels, we start by assessing the original OTF. Usually, OTFs incorporate techniques to guarantee serializability/snapshot isolation for their limited transaction model. We will now investigate what that means for Lemma 5.2.

Assumption: OTF histories are serializable for single-query transactions on a table level.

Proof by contradiction: Assume there are two single-query transactions $t_1, t_2 \in T$ that have a cyclic dependency. Hence, according to Lemma 5.2, such a history is not serializable. This implies using Definition 2.2:

$$t_1 > t_2 \cap t_1 < t_2 \quad (11)$$

OTFs limit themselves to single query transactions:

$$\forall t \in T, |t| = 1 \quad (12)$$

Hence, we can combine this observation with Eq. (11) and get:

$$op_1 \in t_1, op_2 \in t_2, op_1 > op_2 \cap op_1 < op_2 \quad (13)$$

Implying that op_1 and op_2 were executed concurrently and built upon each other's results. There are a few options where this scenario might be possible:

Case 1 ($op_1 = op_2$): If $op_1 = op_2$ this implies in addition to Eq. (12) and Eq. (13):

$$op_2 \in t_1 \cap op_1 \in t_2 \xRightarrow{\text{Eq. (13)}} t_1 = t_2 \quad (14)$$

Hence, they are both the same operation from the same transaction. Therefore, op_1 and op_2 do not break isolation as ordering a single operation is always possible and self-contained.

Case 2 ($op_1 \neq op_2$): If $op_1 \neq op_2$, this ensures that they are both from a different transaction ($t_1 \neq t_2$). Hence, by the atomicity requirement of OTF metadata layers, t_1 and t_2 must have received

two distinct version numbers during the commit. Therefore, we have to take a look at the effects of these transactions on our LH:

Case 2.1 t_1 and t_2 operate on the same table i : Hence, their versions are comparable. Hence, we know from Eq. (11):

$$t_1 > t_2 \Leftrightarrow v_i(t_1) > v_i(t_2) \quad (15)$$

and

$$t_1 < t_2 \Leftrightarrow v_i(t_1) < v_i(t_2) \quad (16)$$

However, by the atomic requests used during version assignments, Eq. (15) and Eq. (16) can not hold simultaneously. Independent transactions always receive a unique version number; hence, t_1 and t_2 can't be independent transactions. Alternatively, if t_1 and t_2 are independent, one must have had access to a version with a higher version number than its own. This is a strict violation of the OTF protocols and, thus, is not valid access.

Case 2.2 t_1 and t_2 operate on the distinct tables i_1, i_2 : Hence, the limitation of OTFs to single-table transactions makes the transactions incomparable. As they exclusively operate on their respective tables, their order is not defined by the OTF:

$$t_1 \not\sim t_2 \Rightarrow op_1 \not\sim op_2 \quad (17)$$

Thus, Eq. (11) does not hold in this case.

Since cases 2.1 and 2.2 are impossible under the restrictions of OTFs, we know that Eq. (11) does not hold for $op_1 \neq op_2$.

Further, in cases 1 and 2, Eq. (11) does not hold. Hence, Eq. (11) contradicts the design and structure of OTFs within their transaction model, implying that OTFs ensure a strict and sequential order among all transactions, making them serializable for single-query and single-table transactions. $\square$

In summary, for all transactions $t_1, t_2 \in T$, OTFs define a sequential order among them if they operate on the table ($tbl(t_1) = tbl(t_2)$) by assigning unique version numbers ($t_1 < t_2 \Leftrightarrow v(t_1) < v(t_2)$). This holds for all single query transactions (Eq. (12): $|t_1| = |t_2| = 1$).

Lemma 5.3. *OTFs ensure by design that there is a sequential order among single-query and single-table transactions on the same table: $\forall t_1, t_2 \in T, |t_1| = |t_2| = 1, tbl(t_1) = tbl(t_2), t_1 < t_2 \Leftrightarrow v(t_1) < v(t_2)$*

5.2 Isolation of LV [R]

LV [R] improves upon Lemma 5.3. It targets lifting the restriction to the OTFs' single-query transactions ($|t_1| = |t_2| = 1$).

Assumption: LV [R] guarantees serializability for single-table, multi-query transactions.

Proof by contradiction: Assume LV [R] produces a non-serializable history for all single-table, multi-query transactions T . Then, according to Lemma 5.2, there is a circular dependency between at least two transactions. Hence:

$$\exists C = \{t_1, t_2, \dots, t_n\} \subset T, n \leq |T|, (\forall i \in \{2, \dots, n\}, t_{i-1} < t_i) \cap t_n < t_1 \quad (18)$$

Since LV [R] limits itself to single-table transactions, we know:

$$\forall t \in T, |tbl(t)| = 1 \quad (19)$$

Following our previous observations, we know that only transactions operation on the same table form an order $\sim$. Hence, for C to

form a cycle, we know:

$$\forall t_1, t_2 \in C, |tbl(t_1)| = |tbl(t_2)| = 1 \cap tbl(t_1) = tbl(t_2) \quad (20)$$

LV [R]'s marker technique ensures that each transaction is contained in a single version (log file/branch) in the respective OTF metadata layer (see Definition 3.2). As the order of transactions and the table version assigned to each transaction are linked, we can transform Eq. (18) for the respective table id to:

$$\begin{aligned} \exists C = \{t_1, t_2, \dots, t_n\} \subset T, n \leq |T|, \\ (\forall i \in \{2, \dots, n\}, v_{id}(t_{i-1}) < v_{id}(t_i)) \cap v_{id}(t_n) < v_{id}(t_1) \end{aligned} \quad (21)$$

By transitivity, we know:

$$v_{id}(t_1) < v_{id}(t_n) \cap v_{id}(t_1) > v_{id}(t_n) \quad (22)$$

However, by the definition of v , it returns a natural number $\mathbb{N}$. Hence, due to the usage of atomic requests leading to distinct and unique version numbers, Eq. (22) does not hold. Hence, we know:

$$(\exists i \in \{2, \dots, n\}, v_{id}(t_{i-1}) > v_{id}(t_i)) \cup v_{id}(t_n) > v_{id}(t_1) \quad (23)$$

This finding effectively breaks the cycle, and C in Eq. (18) can not form a cycle. Instead, it forms a sequential order of transactions where each can only access data from transactions with lower version numbers. As the markers ensure the operations of a transaction are in the same log file:

$$\forall op_1, op_2 \in t \in T, tbl(op_1) = tbl(op_2), v(op_1) = v(op_2) \quad (24)$$

, each operation is ordered by its transaction itself (Eq. (5)), and the order of transactions transitively defines a sequential order. By Eq. (23) and the non-cyclic nature of transactions, there can not be cyclic relations between operations like Eq. (18). Hence:

$$\nexists C = \{t_1, t_2, \dots, t_n\} \subset T, n \leq |T|, (\forall i \in \{2, \dots, n\}, t_{i-1} < t_i) \cap t_n < t_1 \quad (25)$$

This gives us a sequential order among the table's multi-query and single-table transactions. $\square$

Lemma 5.4. *LV [R] guarantees serializability for single-table, multi-query transactions:*

$$\forall t_1, t_2 \in T, tbl(t_1) = tbl(t_2), t_1 < t_2 \Leftrightarrow v(t_1) < v(t_2)$$

5.3 Isolation of LV [CT]

LV [CT] reuses the marker technique to remove Lemma 5.3's restriction on single tables ($\forall t \in T, |tbl(t)| = 1$). As it reuses the marker idea of LV [R], we only prove that LV [CT] ensures that it creates a serializable order of transactions across tables. All other proofs are analogous to LV [R]'s for operations on the same table.

Assumption: LV [CT] provides repeatable reads across tables for multi-table, multi-query transactions.

Direct Proof: LV [CT] sets markers in each accessed table of a transaction. Hence, it reserves a set of table versions within the respective metadata layers. Hence:

$$\forall t \in T, \forall id \in tbl(t), \exists v_{id}(t) \quad (26)$$

By applying Lemma 5.4 per table in T , we see that this orders all operations for each table (as LV [CT] works similarly to LV [R] here):

$$\forall t_1, t_2 \in T, \forall id \in tbl(t_1) \cap tbl(t_2), t_1 \sim t_2 \quad (27)$$

However, unlike LV [R], if a transaction accesses a new table, LV [CT] sets a new marker in the respective metadata layer. Given two concurrent multi-table transactions $t_1, t_2 \in T$, t_2 decides to access a new table id . In this scenario, we have two possibilities:

Case 1: t_1 did not access id yet. In this case, t_1 executed no operations on id . Hence, all of t_1 's operations can be ordered before or after t_2 's marker version on id , depending on the existing relation $t_1 \sim t_2$ on another table. Hence, t_1 is isolated by default. There is no need for LV [CT] to readjust this situation.

Case 2: t_1 already accessed id . Then, a marker for t_1 in id already exists. When t_2 sets a new marker, this automatically forms the relation:

$$t_1 < t_2 \Leftrightarrow v_{id}(t_1) < v_{id}(t_2) \quad (28)$$

If there are previously executed operations in t_1, t_2 , we have three scenarios that may apply here:

Case 2.1: $(tbl(t_1) \cap tbl(t_2)) = \{id\}$. Hence, there is no other table that both t_1 and t_2 have both accessed yet. Thus, the two have no relation ($t_1 \not\sim t_2$). Hence, only their global relation is updated to $t_1 < t_2$, and both can continue. On commit, all of t_1 's operations are logically ordered before t_2 . Hence, t_2 must compare its changes to the respective logs and either commit or abort.

Case 2.2: $\forall z \in (tbl(t_1) \cap tbl(t_2))/id, v_z(t_1) < v_z(t_2)$. In this scenario, both transactions encountered each other in another table beforehand (If they are empty, case 2.1 applies). Hence:

$$(tbl(t_1) \cap tbl(t_2))/id \neq \emptyset \quad (29)$$

In these tables, t_1 and t_2 already hold the newly created relation within id . Hence, this operation does not violate the previous relation $t_1 < t_2$. Thus, LV [CT] automatically upholds this relation, and both transactions can continue normally.

Case 2.3: $\forall z \in (tbl(t_1) \cap tbl(t_2))/id, v_z(t_1) > v_z(t_2)$. Hence, we now have two contradicting relations in different tables. In such a case, LV [CT] triggers a marker shift. As LV [CT] can not rearrange the order within id , its marker shift frees all markers in all other tables and places a new marker. This effectively inverts the relation in such tables.

Lemma 5.5. *LV [CT]'s marker shift inverts the relations between itself and transactions depending on it within the affected tables.*

Hence, with the relation found in id and after a marker shift, LV [CT] guarantees:

$$\forall z \in tbl(t_1) \cap tbl(t_2), v_z(t_1) < v_z(t_2) \quad (30)$$

This fixes the global relation between those two transactions. Since Cases 2.1, 2.2, and 2.3 ensured consistent ordering between transactions, LV [CT] provides an order among the tables.

However, LV [CT] is not serializable due to the marker shift: A transaction can never read dirty data due to the marker replacement during the commit. To prevent Non-repeatable reads, LV [CT] remembers the old read version of a table and will reuse it if a similar read is triggered after a marker shift. This operation will return the same result as before, but the system must check during the transaction's commit to see if it is viable. If a log entry of a preceding marker contains changes to the file in the read value, the marker-shifted transaction must abort, upholding consistency. Despite that, phantom reads can occur: If a transaction requests a

range of tuples, it usually accesses multiple data files in the OTF table. As LV [CT] only remembers previous reads, it can not predict any changes incorporated by a marker shift (if a transaction that was previously dependent and now became preceding) belongs to a range query. Hence, for LV [CT], the same query might look like a different operation that will work with the new changes incorporated. This circumstance, however, might yield a different result. Hence, because LV [CT] defines a structure between dependent transactions and avoids dirty and non-repeatable reads, it provides repeatable reads globally. $\square$

Lemma 5.6. *LV [CT] guarantees repeatable reads globally.*

5.4 Isolation of LV [R, CT]

To improve Lemma 5.6, we want to point out the combination LV [R, CT]. Here, a marker shift utilizes LV [R]'s sublog structure to revert the transaction to its first operation on a table of the set of affected tables by the marker shift. Afterward, LV [R, CT] executes all reverted operations using the new properties with each table. Theoretically, this can be understood as an abort with an automatic creation of a new, adapted transaction. Hence, the transaction committing in the end will continuously operate on the same version combination across all tables, avoiding phantom reads. Hence:

Lemma 5.7. *LV [R, CT] guarantees serializability for multi-table, multi-query transactions:*

$$\forall t_1, t_2 \in T, t_1 < t_2 \Leftrightarrow v(t_1) < v(t_2)$$

5.5 Isolation of LV [I]

LV [I] focuses on giving the same isolation level as Lemma 5.7 using a global commit protocol.

Assumption: LV [I] guarantees serializability

Proof by contradiction: Assume LV [I] is not serializable. Then, according to Lemma 5.3, two transactions $t_1, t_2 \in T$ can not be ordered sequentially:

$$\exists t_1, t_2 \in T, t_1 \neq t_2, t_1 < t_2 \cap t_1 > t_2 \quad (31)$$

Since LV [I] takes version assignment to its global version log, both transactions received their table versions. Since t_1 is able to read t_2 's data, we:

$$\exists id \in tbl(t_1) \cap tbl(t_2), v_{id}(t_2) < v_{id}(t_1) \quad (32)$$

Thus, the global version log must have previously known $v_{id}(t_2)$. Hence, t_2 must have passed the validation phase, followed by committing in all its accessed tables before making its changes globally available. However, t_2 observed t_1 's changes, implying:

$$\exists id_2 \in tbl(t_1) \cap tbl(t_2), v_{id_2}(t_2) > v_{id_2}(t_1) \quad (33)$$

Hence, t_2 received a version $v_{id_2}(t_2)$ larger than $v_{id_2}(t_1)$ from the global version log. Hence, t_1 must have passed validation and committed before t_2 . This version assignment is impossible with the atomic validation phase of LV [I]. Transactions pass through it sequentially and end it by committing or aborting. Hence, there can not be a circular dependency between t_1 and t_2 .

The only other option for a discrepancy in version assignment to occur is:

$$\exists t_3 \in T, t_3 < t_2 \cap t_3 < t_1 \quad (34)$$

Hence, another transaction was committed before t_1 and t_2 . Thus, this commit changed the table version in the global version log. If w.l.o.g. t_1 gets its version assignment before t_3 commits and t_2 gets a version assignment after t_3 commits, their versions differ. However, this will imply:

$$\exists id_2 \in tbl(t_1) \cap tbl(t_2), v_{id_2}(t_2) > v_{id_2}(t_1) \quad (35)$$

But as version numbers in the global version log can only be incremented:

$$\nexists id \in tbl(t_1) \cap tbl(t_2), v_{id}(t_2) < v_{id}(t_1) \quad (36)$$

Hence, LV [I] does not create a circular dependency. Further, t_1 will be aborted after validation as it cannot claim its assigned version $v_{id_2}(t_1)$. Hence, LV [I] defines a global order of transactions with no circular dependencies. Due to the fixed versions of all table transactions received at the beginning, they will continuously operate on identical versions, preventing dirty, non-repeatable, and phantom reads. If a transaction is seen to be not compatible with the currently announced version combination during LV [I]'s validation phase, it will abort. Ensuring that the commit order of LV [I] matches a global sequential execution. This makes LV [I] serializable. $\square$

Lemma 5.8. *LV [I] guarantees serializability for multi-table, multi-query transactions:*

$$\forall t_1, t_2 \in T, t_1 < t_2 \Leftrightarrow v(t_1) < v(t_2)$$

Moreover, if we relax the hidden_version to a larger area and let read-only transactions pass validation without a check, we skip the check on the transactions read sets, isolating the LV [I] snapshot.

Lemma 5.9. *LV [i] with a relaxed hidden_version guarantees snapshot isolation for multi-table, multi-query transactions.*

6 Durability of OTFs and LV

All OTFs we investigated ensured that write transactions complete their data upload before committing. Since object stores are durable storage services, they can rely on them to guarantee durability as soon as the data upload succeeds. As all LV features do not change this behavior, this also holds for LV. Hence:

Lemma 6.1. *OTFs and LV ensure durability by using durability guarantees of the object store.*

Received 1 March 2026